

User Management CRUD API

Backend Development Project 2 – Database Integration

Submitted By:

Nosheen Afzal

Technologies Used:

Node.js | Express.js | MongoDB | Mongoose

Date: June 2026

Table of Contents

Introduction -----	3
Objectives -----	3
Technologies Used -----	3
Project Structure -----	3
Database Design -----	4
API Endpoints -----	4
Testing & Results -----	5
Conclusion -----	8

Introduction

The User Management CRUD API is a RESTful backend application developed using Node.js, Express.js, and MongoDB.

The application demonstrates database integration by performing Create, Read, Update, and Delete (CRUD) operations on user records while ensuring data persistence and preventing duplicate entries.

Objectives

The objectives of this project are:

- Connect an API to a database
- Design a User schema
- Implement CRUD functionality
- Store data permanently in MongoDB
- Prevent duplicate user entries
- Practice NoSQL database operations using Mongoose

Technologies Used

Technology	Purpose
Node.js	Runtime Environment
Express.js	Backend Framework
MongoDB	NoSQL Database
Mongoose	ODM for MongoDB
Postman	API Testing
VS Code	Development Environment

Project Structure

Task-2-Nosheen-Afzal/

```
|
|
├── models/
|   └── User.js
|
|
├── routes/
```

```
|   └── userRoutes.js
|
|── screenshots/
|
|── .gitignore
|── package.json
|── package-lock.json
|── server.js
|── README.md
└── Backend Task 2.pdf
```

Database Design

User Schema

```
{
  name: String,
  email: String,
  age: Number
}
```

Field Description

Field	Type	Description
name	String	User Name
email	String	Unique Email
age	Number	User Age

Validation

- Name is required
- Email is required
- Email must be unique
- Age is required

API Endpoints

Create User

POST

/api/users

Sample Request:

```
{
  "name": "Nosheen",
  "email": "nosheen@gmail.com",
  "age": 22
}
```

Get All Users

GET

/api/users

Get User By ID

GET

/api/users/:id

Update User

PUT

/api/users/:id

Delete User

DELETE

/api/users/:id

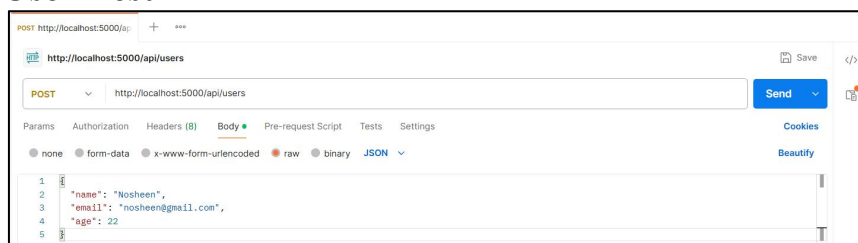
Testing & Results

MongoDB Connection

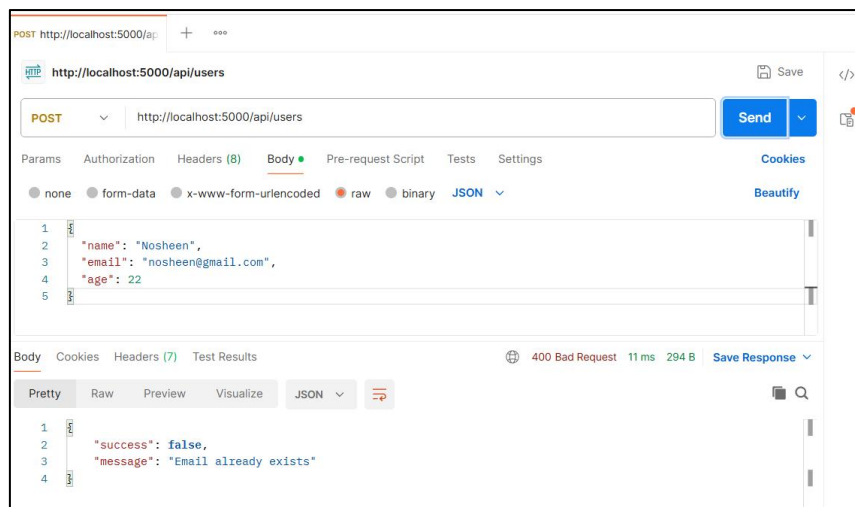
```
user-management-crud-api> npm run dev

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: ~ auth for agents [www.vestaauth.com]
MongoDB Connected
Server running on port 5000
```

Create User Test



Duplicate Email Validation



POST http://localhost:5000/api/users

Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Nosheen",
3   "email": "nosheen@gmail.com",
4   "age": 22
5 }
```

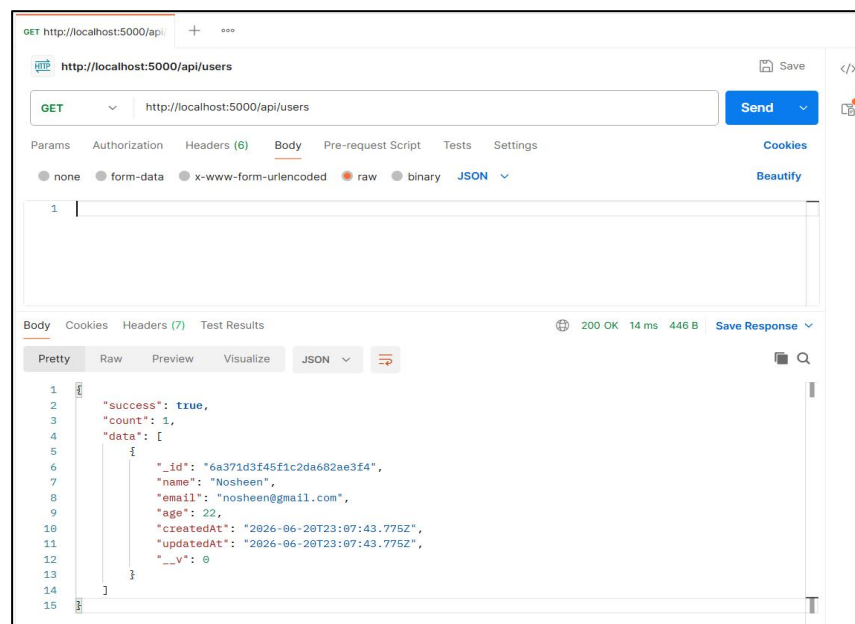
Body Cookies Headers (7) Test Results

400 Bad Request 11 ms 294 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "success": false,
3   "message": "Email already exists"
4 }
```

Get All Users



GET http://localhost:5000/api/users

Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

1

Body Cookies Headers (7) Test Results

200 OK 14 ms 448 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "success": true,
3   "count": 1,
4   "data": [
5     {
6       "_id": "6a371d3f45f1c2da682ae3f4",
7       "name": "Nosheen",
8       "email": "nosheen@gmail.com",
9       "age": 22,
10      "createdAt": "2026-06-20T23:07:43.775Z",
11      "updatedAt": "2026-06-20T23:07:43.775Z",
12      "__v": 0
13    }
14  ]
15 }
```

Get User By ID

The screenshot shows a REST client interface with a GET request to `http://localhost:5000/api/users/6a371d3f45f1c2da682ae3f4`. The response is a JSON object indicating success and returning user details.

```
1 {
2   "success": true,
3   "data": {
4     "_id": "6a371d3f45f1c2da682ae3f4",
5     "name": "Nosheen",
6     "email": "nosheen@gmail.com",
7     "age": 22,
8     "createdAt": "2026-06-20T23:07:43.775Z",
9     "updatedAt": "2026-06-20T23:07:43.775Z",
10    "__v": 0
11  }
12 }
```

Update User

The screenshot shows a REST client interface with a PUT request to `http://localhost:5000/api/users/6a371d3f45f1c2da682ae3f4`. The request body contains the updated name. The response is a JSON object indicating success and returning the updated user details.

```
1 {
2   "name": "Nosheen Afzal"
3 }
```

```
1 {
2   "success": true,
3   "message": "User updated successfully",
4   "data": {
5     "_id": "6a371d3f45f1c2da682ae3f4",
6     "name": "Nosheen Afzal",
7     "email": "nosheen@gmail.com",
8     "age": 22,
9     "createdAt": "2026-06-20T23:07:43.775Z",
10    "updatedAt": "2026-06-20T23:13:52.582Z",
11    "__v": 0
12  }
13 }
```

Delete User

The screenshot shows a REST client interface with a DELETE request to `http://localhost:5000/api/users/6a371d3f45f1c2da682ae3f4`. The response is a JSON object indicating success.

```
1 {
2   "success": true,
3   "message": "User deleted successfully"
4 }
```

Conclusion

This project successfully implemented a RESTful API integrated with MongoDB using Mongoose. CRUD operations were developed and tested using Postman. The application demonstrates database connectivity, data persistence, schema design, and duplicate-entry prevention.